



Kubernetes Technical Briefing

Module 2:
Azure Kubernetes Service



Agenda

- Introduction to AKS
- Node Pools / Cluster Autoscaler
- Authentication & Authorization
- Kubernetes RBAC and Azure AD
- Networking
- AKS Integrations

Introduction to Azure Kubernetes Service



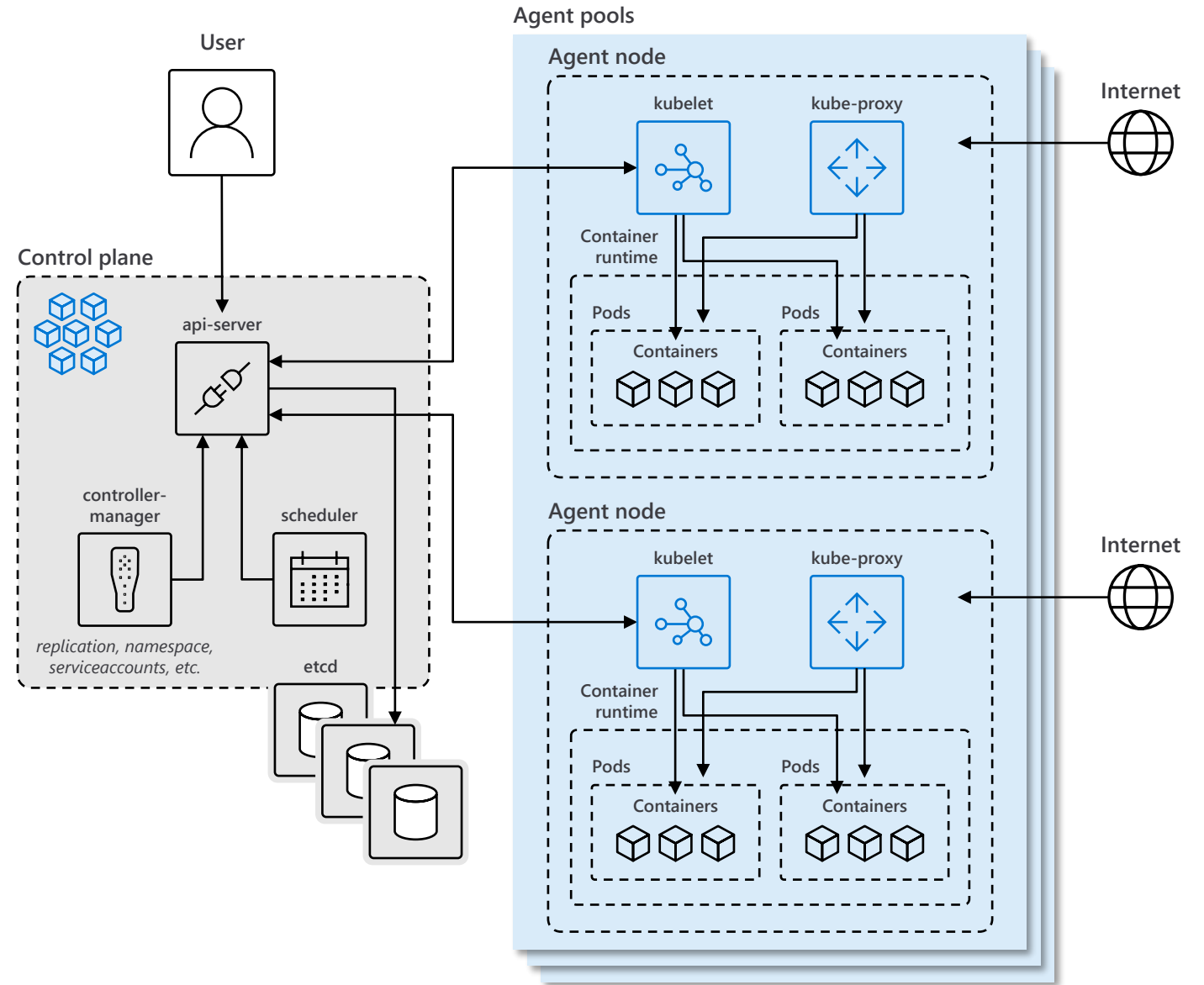
Kubernetes is complex

1. Control Plane: manages the agent nodes and the pods in the cluster

- **api-server**: front end of the Kubernetes control plane; exposes Kubernetes API
- **controller-manager**: runs the controller processes
- **scheduler**: tracks newly created pods and selects node to run them on
- **etcd**: stores the state of the cluster (config, running workloads status, etc.)

2. Agent nodes: run your application workloads

- **Pods**: a collection of containers co-located on a single machine
- **kube-proxy**: a network proxy that runs on each node in a cluster
- **kubelet**: agent that runs on each node in a cluster; ensures containers are running in a pod
- **Containers**: software responsible for running containers



Managed Kubernetes handles the complexity for you

Automated upgrades, patches

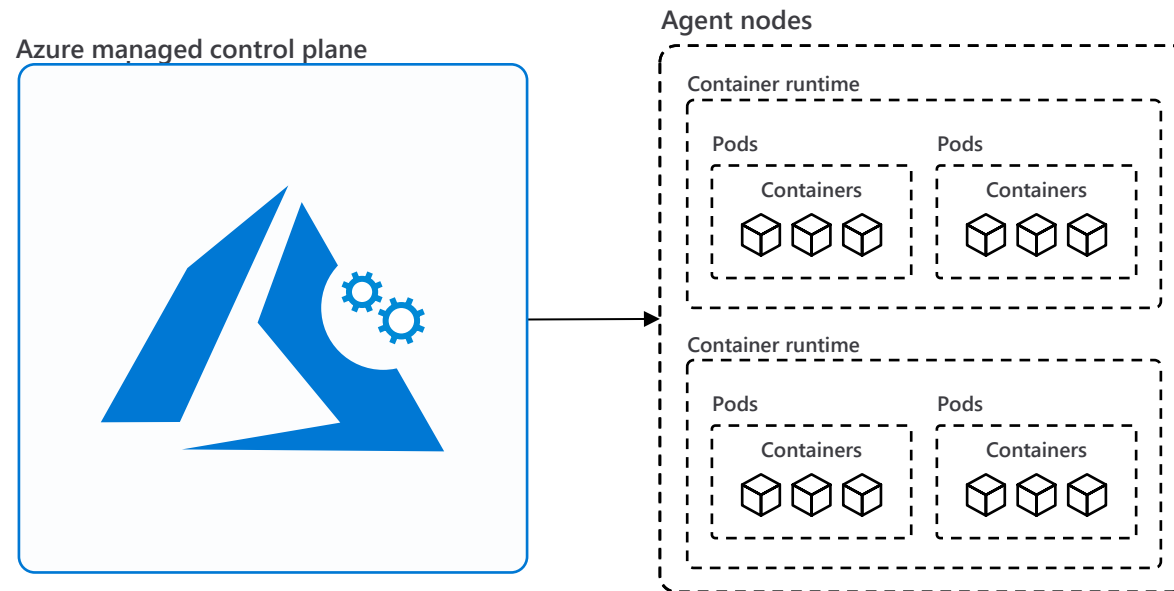
High reliability, availability

Easy, secure cluster scaling

Self-healing

API server monitoring

At no charge*



*Higher SLA guarantees available as an optional uptime SLA paid feature

But...

Managed Kubernetes is just a starting point

Things to consider as you adopt management Kubernetes

Security

How do you manage access to clusters?
What about managing sensitive info?



Developer tools

What does the developer toolchain look like?



Platform management

Should we use a single cloud provider or multiple? Own our data center?
Consider a hybrid setup?



DevOps

What processes do we need for a smooth operation?



Managed Kubernetes on Azure

Security

- Azure Active Directory
- Azure Policy
- Azure Security Center
- Azure Key Vault

Developer tools

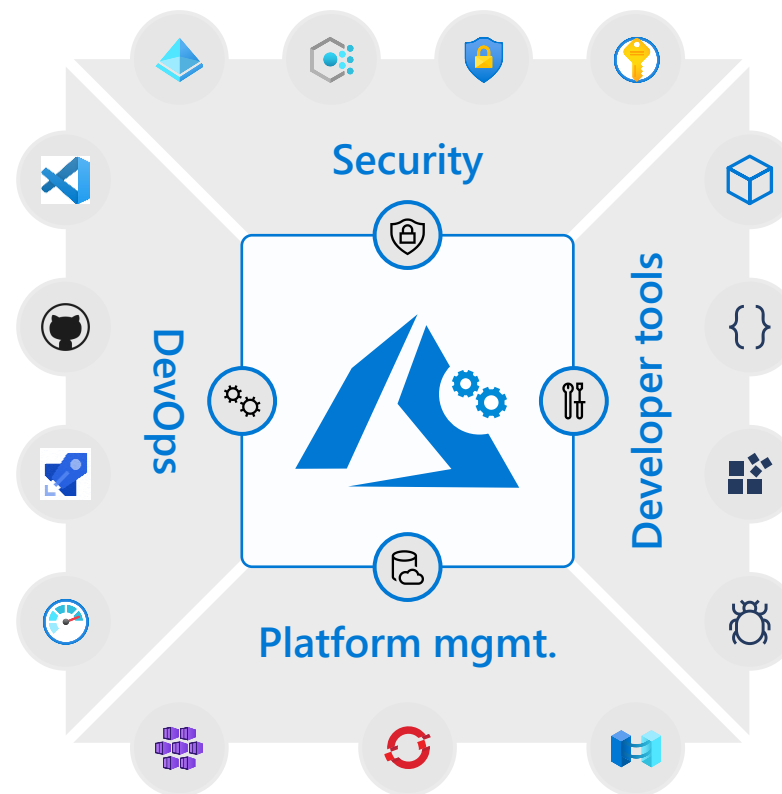
- IDE container support
- Source code repository
- Registry supporting Helm
- Microservice debugging

Platform management

- Azure Kubernetes Service
- Azure Red Hat OpenShift
- Azure Arc-enabled Kubernetes

DevOps

- Visual Studio Code
- GitHub
- Azure Pipelines
- Azure Monitor



Work how you want with opensource tools and APIs

	Development	DevOps	Monitoring	Networking	Storage	Security
Take advantage of services and tools in the Kubernetes ecosystem	   Virtual kubelet  CNAB	    HashiCorp 	 Prometheus  fluentd  Grafana  DATADOG  	 CNI Networking  TIGERA	  portworx  ceph OpenEBS  GlusterFS  STORAGEOS	 Twistlock  aqua  Qualys  sysdig
Leverage 100+ turn-key Azure services	 Visual Studio Code  GitHub	 Azure Pipelines  ARM  Container Registry	 Azure Monitor	 Azure VNET  Service Mesh Interface	 Azure Storage  Azure Cosmos DB	 Azure Policy  AAD  ASC  Key Vault

Azure Kubernetes Service (AKS)

- AKS automates provisioning, upgrading, monitoring, and scaling with, a simpler development-to-production experience, and enterprise-grade security and governance.
- By providing Kubernetes as a managed service, AKS abstracts the complexity and operational overhead of managing Kubernetes.
- AKS includes 2 tiers standard and free.
 - Premium: Mission Critical, 2 years of AKS Version Support (0.60 per cluster per hour)
 - Standard tier: large clusters, 5000 nodes, SLA, HA (0.10 per cluster per hour)
 - Free: smaller clusters, ideal for experimentation, can go up to 1000, no SLA
- You have full control over the cluster's cost based on the size and type resources you choose to add to it and how available do you need your cluster to be.

Installing AKS

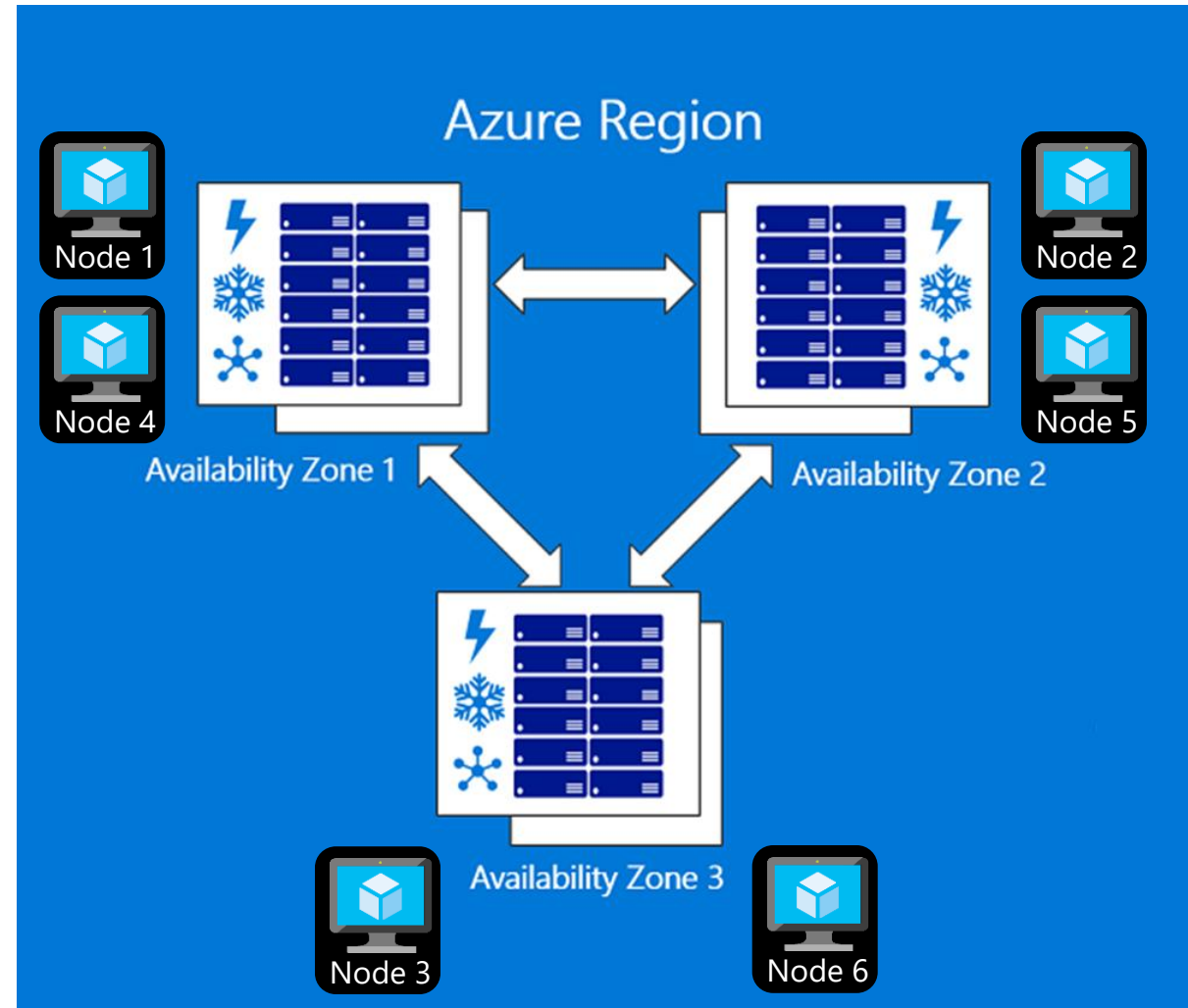
- There are multiple ways to create/upgrade Azure Kubernetes Service:
 - **Azure Portal** – A web-based UI that allows you create AKS clusters with the most commonly used settings (not all settings are available in the Azure Portal). Good for training and creating development clusters.
 - **Azure CLI** – A cross-platform command-line interfaces that allows you to configure ALL AKS features and settings. Use when creating production clusters.
 - **Bicep/Terraform** – Infrastructure as code tools that lets you *declaratively* define the configuration of your clusters. Use to deploy multiple clusters and supporting services.

Kubernetes Version

- Kubernetes is an open-source application hosted on GitHub.
- Major and minor releases are released frequently.
- When creating Azure Kubernetes Service cluster, you need to decide which version of Kubernetes you want to install as your control plane.
- The default version selected in the Azure Portal is **2** releases back from the latest stable (non-preview) release.
- You can upgrade your cluster to a newer version later.
- The nodes must be running a version of Kubernetes that is equal to or less than the current version on the control plane.

Availability Zones

- **Availability Zone** - Unique physical locations within a region. Each zone is made up of one or more datacenters equipped with independent ***power, cooling, and networking***.
- Zone-redundant services distribute your applications across Availability Zones to protect from single-points-of-failure.



AKS Availability Zones are NOT Available in all Regions

- **Azure Region** - A set of datacenters deployed within a latency-defined perimeter and connected through a dedicated regional low-latency network.
- AKS does NOT support **Availability Zones** in all regions.
- It is recommended that production clusters be deployed to “recommended” regions that support Availability Zones.
- Clusters using Availability Zones come with a 99.95% uptime SLA (99.5% otherwise).
- There’s a \$0.10 fee per hour, per cluster when Availability Zones are used.

Node Pools



Cluster Node Pools

- A **Node Pool** is a set of Nodes (VMs) with the exact same configuration.
- In AKS, each Node Pool maps to a **Virtual Machine Scale Set**.
- Every AKS cluster contains at least one Node Pool.
- A default Node Pool is created when a new AKS cluster is provisioned.
- The default Node Pool is designated as a **System** Node Pool, for running Kubernetes utilities.
 - System Nodes should not be confused with Master Nodes, which control the Kubernetes cluster and are hidden.
 - You have no control over the configuration and number of Master Nodes.
- While you can schedule your workloads on system nodes, it's recommended that you create a separate **User** Node Pool to run your Pods.

Types of Node Pools

System Node Pool

Used by system utilities like **CoreDNS** and **Metrics Server**. System Node Pools can only use the Linux OS.

Linux User Node Pool

Pool of VMs that can run user workloads requiring Linux

Windows User Node Pool

Pool of VMs that can run user workloads requiring Windows

Spot Node Pool

A User Node Pool that consists of Spot VM Instances. Azure can reclaim these VMs at any time if it needs their compute power. Spot VMs are up to 90% cheaper than regular VMs.

Virtual Nodes

Dynamic nodes created as wrappers for Azure Container Instances. See [limitations](#).

Use Labels and Node Selectors to Schedule Workloads

Leverage well-known, prepopulated node labels and node selectors to ensure your workloads are scheduled in the correct node pools.

System Node Pool

```
nodeSelector:  
  kubernetes.azure.com/mode: system
```

Linux User Node Pool

```
nodeSelector:  
  kubernetes.azure.com/mode: user  
  kubernetes.io/os: linux
```

Windows Node Pool

```
nodeSelector:  
  kubernetes.io/os: windows
```

User Spot Node Pool

```
nodeSelector:  
  kubernetes.azure.com/mode: user  
  kubernetes.azure.com/scalesetpriority: spot  
  kubernetes.io/os: linux
```

Virtual Nodes

```
nodeSelector:  
  kubernetes.io/role: agent  
  kubernetes.io/os: linux  
  type: virtual-kubelet  
tolerations:  
- key: virtual-kubelet.io/provider  
  operator: Exists
```

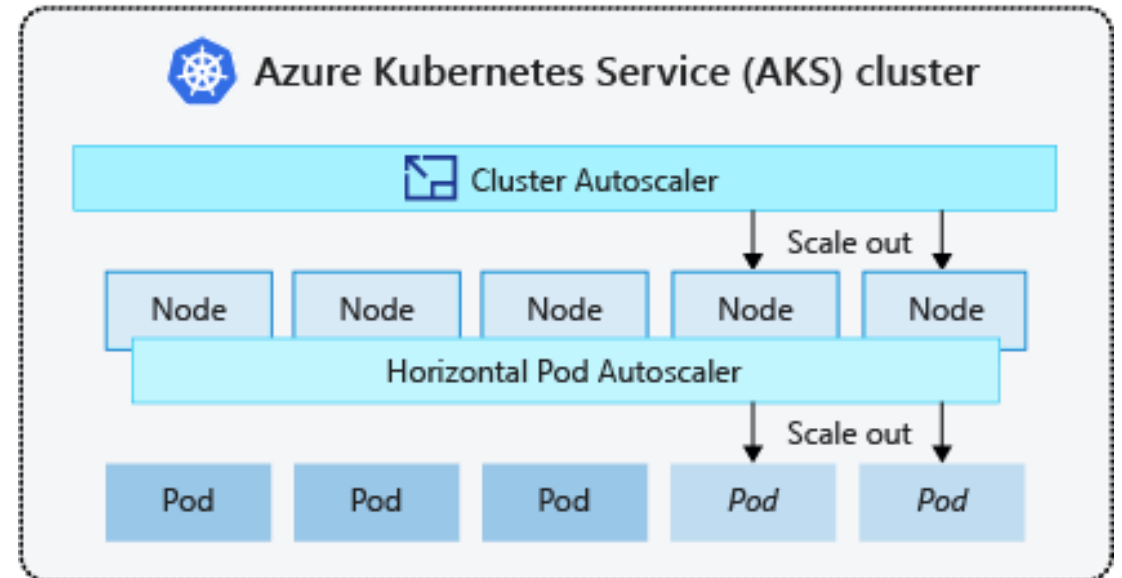
You can also automatically add custom labels to every node in a node pool and implement your own node selection criteria.

Right-Sizing VM Sizes

- The number and size of VMs in a clusters will depend on your system requirements.
- Use a *smaller* VM size for the **System Node Pool** to minimize costs. For example, using DS2_v2 nodes with the default 512 GB OS disk are enough for most system workloads.
- For **User Node Pools**, use *larger* VM sizes to pack the maximum number of pods on a node. This will minimize the footprint of services that run on all nodes, such as monitoring and logging.
- Deploy at least 2-3 nodes per node pool. That way, the workload will have a high availability pattern with multiple replicas.
- Use the **Cluster Autoscaler** to have AKS automatically adjust node counts based on demand instead of always having enough nodes available to handle peak loads.
- When planning capacity for your cluster, assume that your workload can consume up to 80% of each node; the remaining 20% is reserved for AKS services.
- If the node sizes/counts don't meet your needs, you can reconfigure them at any time.

Cluster Autoscaler

- The **Cluster Autoscaler** watches for pods in your cluster that can't be scheduled because of resource constraints.
- When demand goes up, the number of nodes in a node pool is increased to meet the application demand.
- When demand goes down, the autoscaler automatically deletes nodes from the node pool.
- Each node pool can have different scaling options or manual scaling.



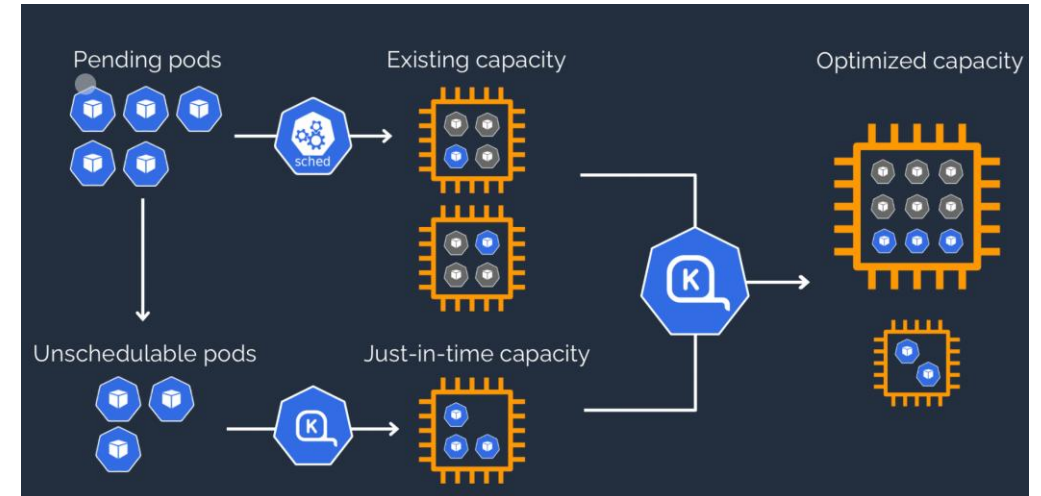
Cluster Autoscaler Customization

In addition to Min/Max number of nodes, the **Cluster Autoscaler** can be configured for fine-grain control of its operations. Here are just some of the options available:

Setting	Description	Default value
scan-interval	How often cluster is reevaluated for scale up or down	10 seconds
scale-down-delay-after-add	How long after scale up that scale down evaluation resumes	10 minutes
scale-down-delay-after-delete	How long after node deletion that scale down evaluation resumes	scan-interval
scale-down-unneeded-time	How long a node should be unneeded before it is eligible for scale down	10 minutes
expander	Type of node pool expander to be used in scale up. Possible values: most-pods, random, least-waste, priority	Random
Many more...		

Scaling Node Auto provisioning (NAP) Preview

- NAP (Preview) is based on Karpenter
 - Provider is [open source](#)
- Deploys and configures and manages Karpenter on AKS
- Karpenter is a CNCF Autoscaling project.
- Limitations
 - Only Supported by Azure CNI Overlay powered by Cilium
 - Can't enabled in a cluster with cluster autoscaler enabled
 - [Additional Limitations](#)
- You can deploy Karpenter stand alone vs the NAP managed version



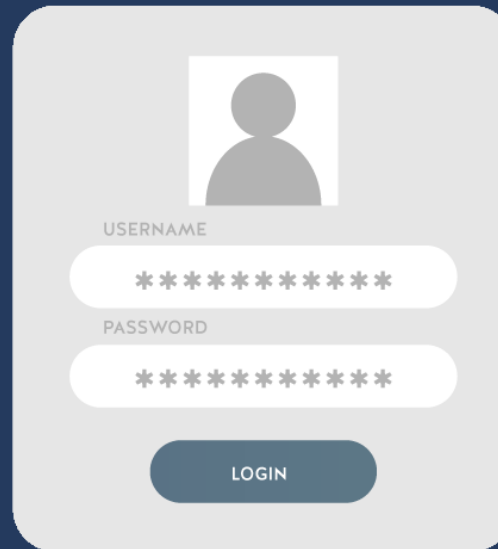
NodePools and Autoscaler

Demo

Authentication & Authorization

3

AUTHENTICATION

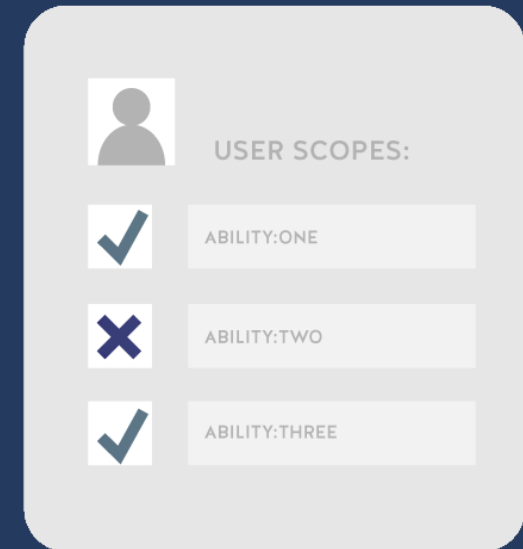


A diagram of an authentication form. At the top is a placeholder for a user profile picture. Below it are two input fields: 'USERNAME' and 'PASSWORD', both containing masked text (asterisks). At the bottom is a 'LOGIN' button.

WHO ARE YOU?

VS

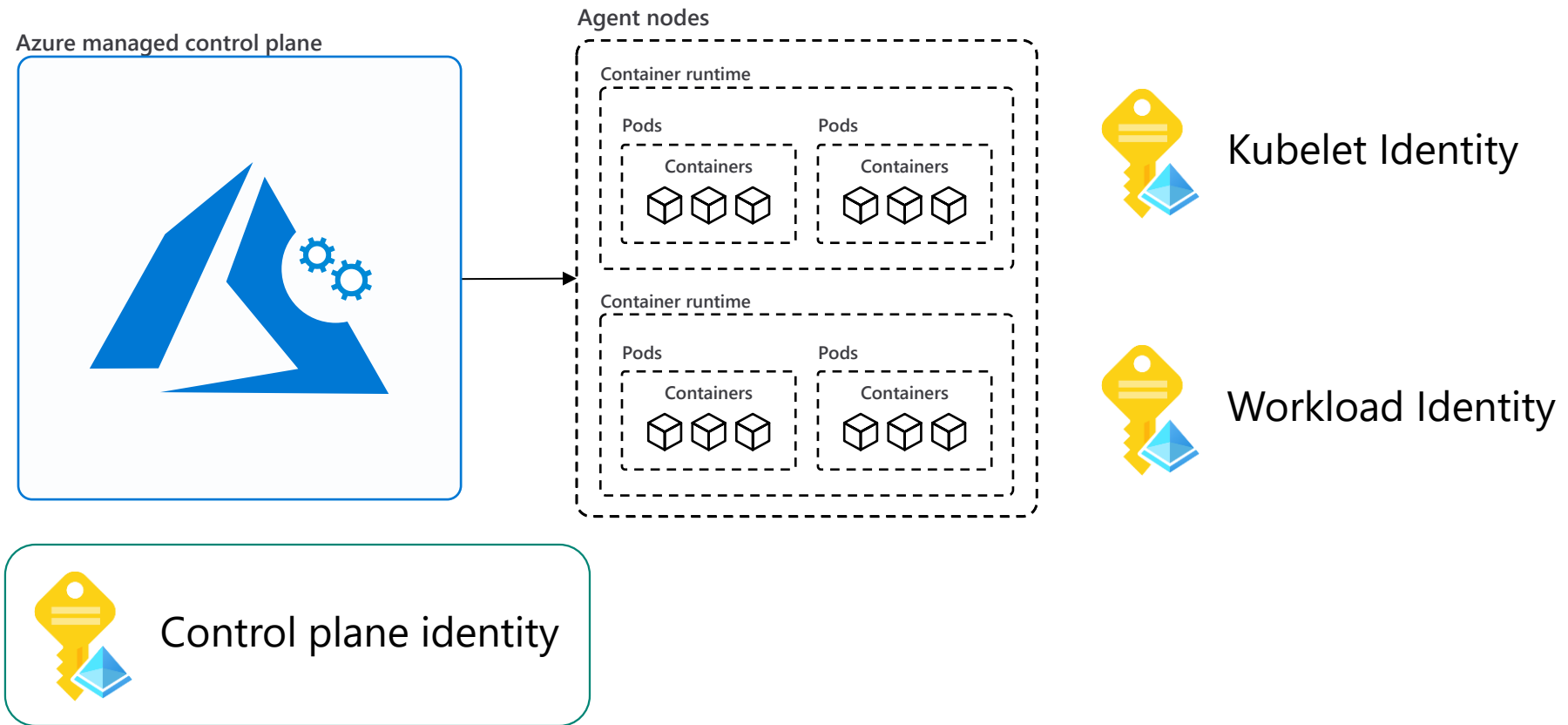
AUTHORIZATION



A diagram of an authorization form. At the top is a placeholder for a user profile picture. Below it is the text 'USER SCOPES:'. There are three rows, each with a checkmark or an 'X' icon and a label: 'ABILITY:ONE' (checked), 'ABILITY:TWO' (unchecked), and 'ABILITY:THREE' (checked).

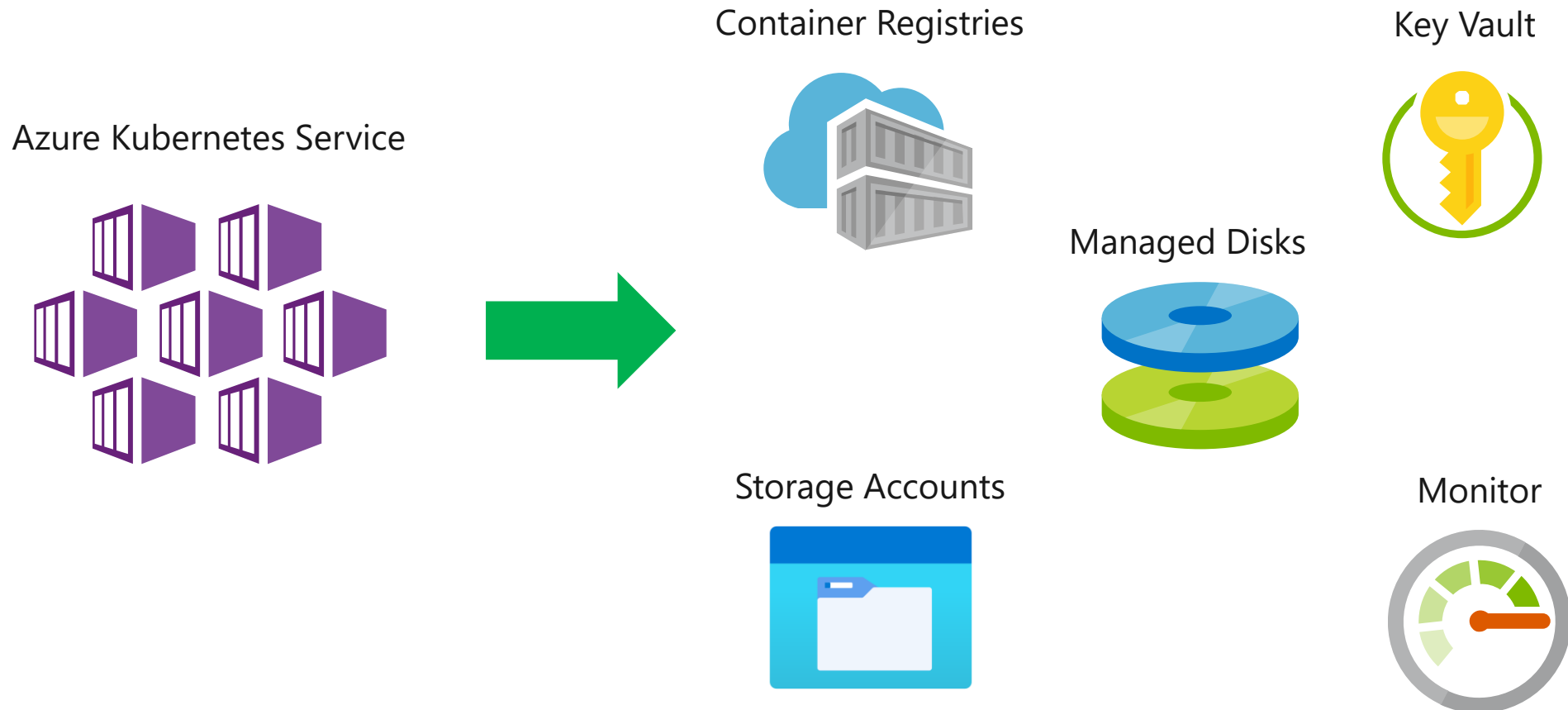
CAN YOU DO THAT?

AKS Service Permissions



Authentication and Authorization from Azure Kubernetes Service to Azure Resources

An Azure Kubernetes Service cluster to be given permissions to access other Azure resources:



Type of Service Principals

There are 2 types of Service Principals in Azure:

- **Application** - This type of service principal is the local representation of a global **application** object in a single tenant or directory. The service principal object defines what the app *can do in the specific tenant, who can access the app, and what resources the app can access*.
- **Managed Identity** - Managed identities eliminate the need for developers to manage credentials. Managed identities provide an identity for applications to use *when connecting to **resources** that support Azure AD authentication*. Service principals representing managed identities can be granted access and permissions but cannot be updated or modified directly.

Azure Role Based Access Control

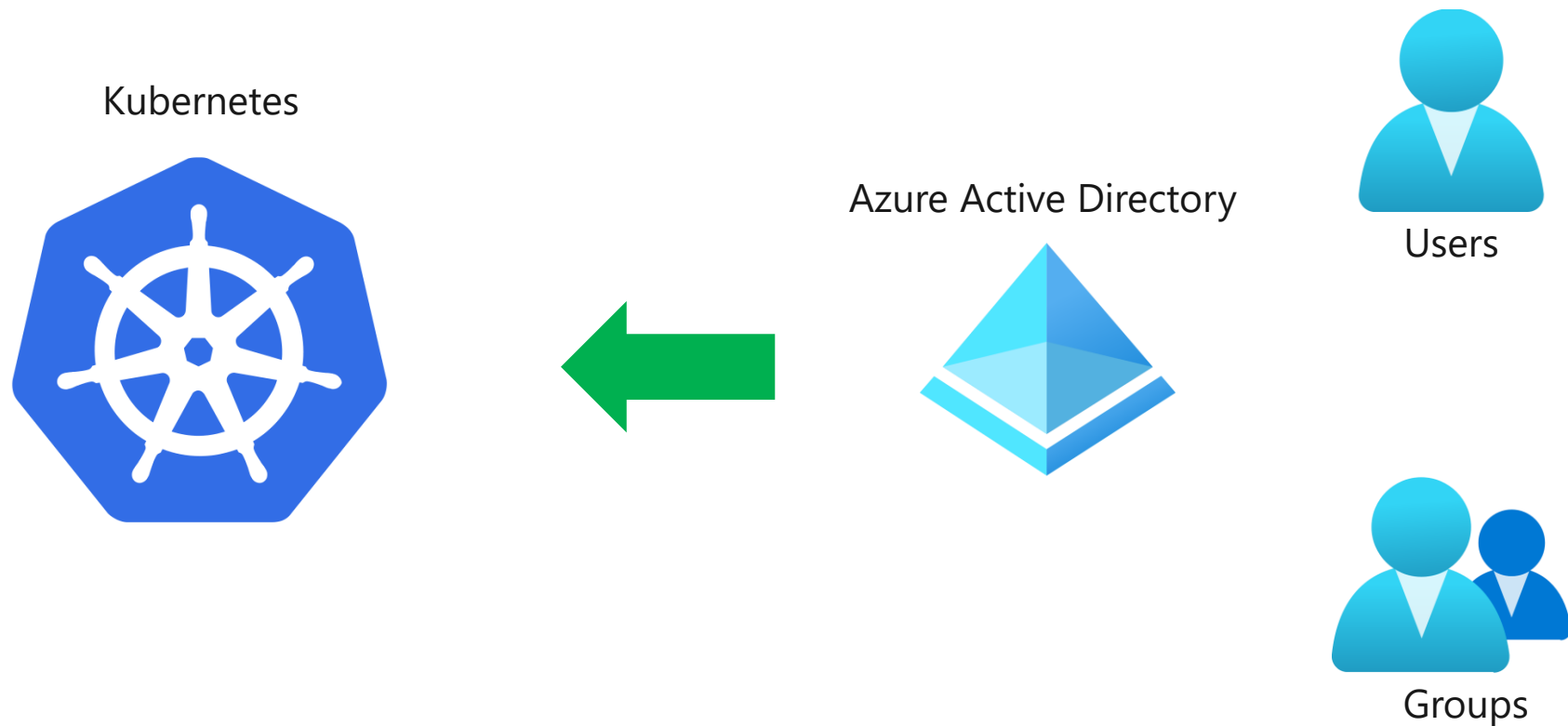
- Azure uses **Role-Based Access Control** (RBAC) to associate permissions with Service Principals.
- Roles common to most resources:
 - **Owner** – Can perform any action, including assign RBAC to other objects
 - **Contributor** – Can perform any action *except* assigning RBAC to others.
 - **Reader** – Can only perform read actions of a resource.
- There are also resource-specific roles available to assign to AKS:
 - **AcrPull** – The cluster's ability to pull images from the specified Azure Container Registries.
 - **Key Vault Secrets User** – The cluster's ability to read secret contents from a Key Vault.
 - **Monitoring Metrics Publisher** - The cluster's ability to send metrics to Azure Monitor.
 - **Network Contributor** - The cluster's ability to control the spoke virtual network.
- If none of the built-in RBAC roles match requirements, you can create custom roles to meet your needs.

Azure Service Principals and Azure RBAC Demo

Manage service principals in the Azure Portal

Authentication and Authorization into Kubernetes

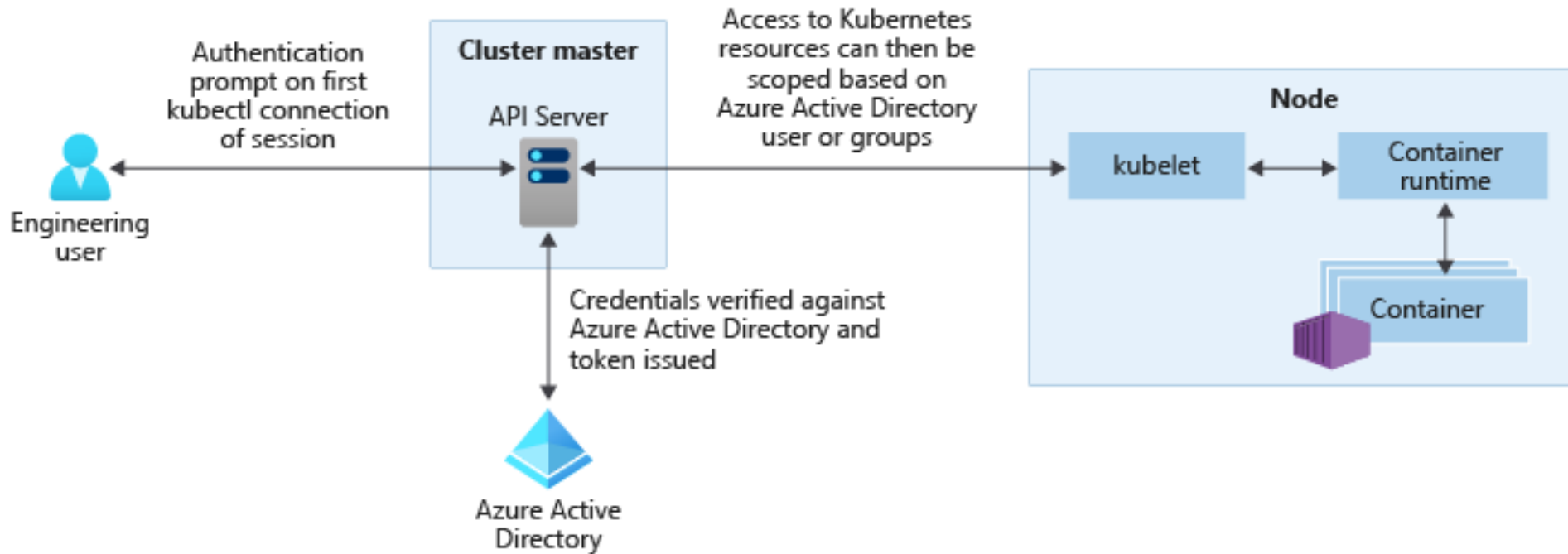
External users/groups need to be given permissions to access a Kubernetes cluster.



Kubernetes User Access

- Kubernetes does not have objects which represent normal user accounts.
- Normal users cannot be added to a cluster through an API call.
- Kubernetes supports several strategies to support **external** authentication.
 - Static Passwords or Tokens
 - X.509 Certificates
 - ***Single Sign-On using OpenID***
 - Authentication Proxy
 - Webhook Token Authentication
- Azure Entra ID can be used to provide authentication to AKS clusters with OpenID Connect.
- You can use Entra ID Users and Groups to control access to AKS clusters.

Entra ID integration



Kubernetes RBAC

- Kubernetes also uses Role-based access control (RBAC) to regulate access to internal resources based on the roles bound to users/groups/service accounts.
- Kubernetes RBAC is enabled by default. Disabling RBAC is **not recommended**.
- Use the `--enable-aad` option to have Entra ID manage AKS authentication.
- Once enabled, Entra ID integration cannot be disabled.
- Kubernetes RBAC works with Azure AD as follows:
 - A user sends the `az aks get-credentials` command to get the credentials of the cluster.
 - Entra ID will authenticate the user's identity against the Azure roles that are allowed to get cluster credentials.
 - The user's token and email is passed to the cluster.
 - The user/group is bound to a Kubernetes **Role**, which defines the scope and actions that are allowed on various Kubernetes resources.

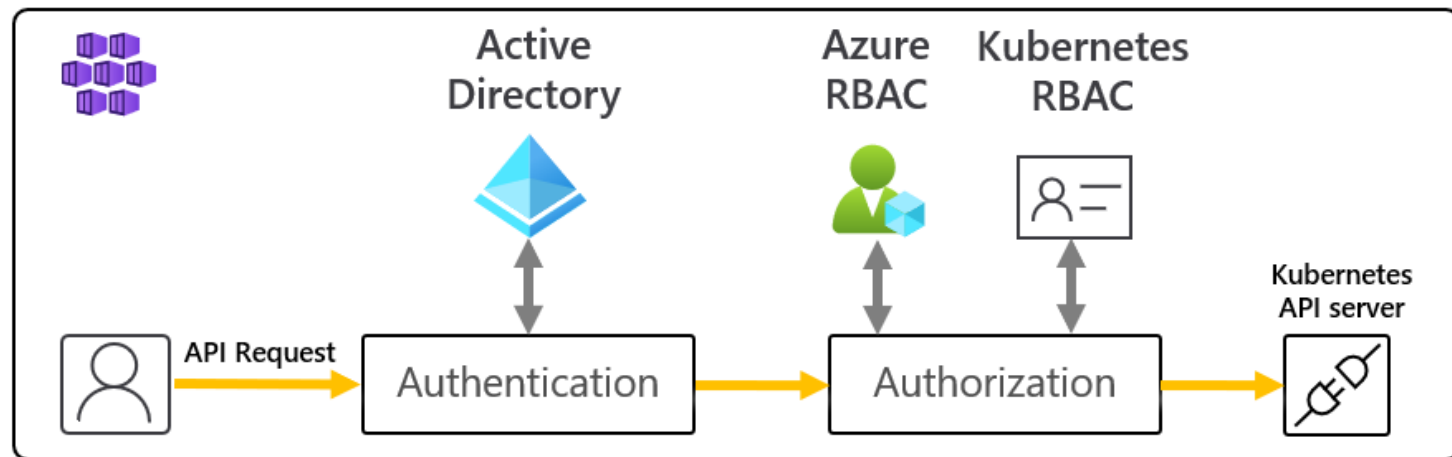
Azure AD with Kubernetes RBAC

Demo

Azure RBAC

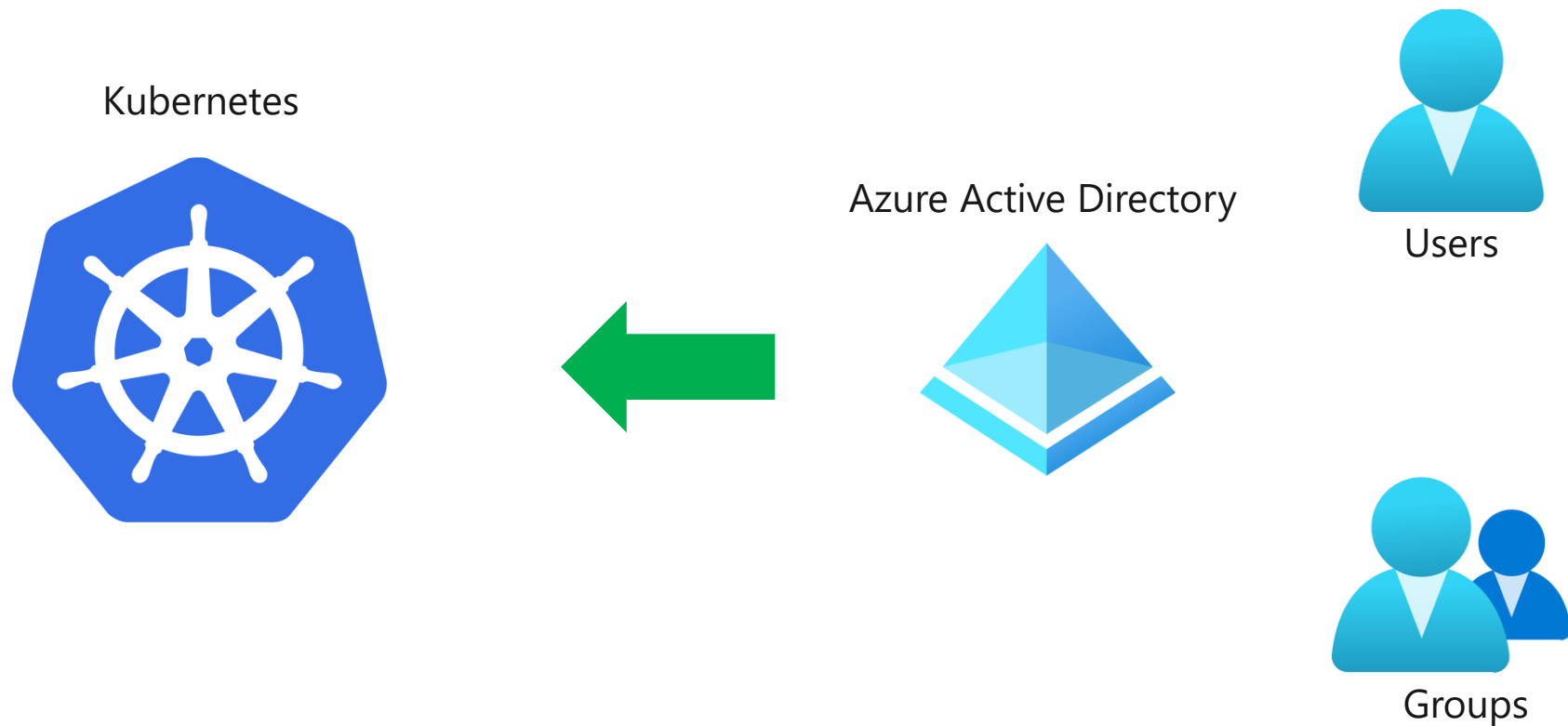
You also have the option of use Azure RBAC, instead of Kubernetes RBAC, to manage authorization within the cluster.

Use the `--enable-azure-rbac` option to use Azure AD RBAC authorization.



Authentication and Authorization into Kubernetes

External users/groups need to be given permissions to access a Kubernetes cluster.



Azure Role Based Access Control

Azure uses ***Role-Based Access Control*** (RBAC) to associate permissions with Service Principals.

Roles common to most resources:

- **Owner** – Can perform any action, including assign RBAC to other objects
- **Contributor** – Can perform any action *except* assigning RBAC to others.
- **Reader** – Can only perform read actions of a resource.

There are also resource-specific roles available to assign to AKS:

- **AcrPull** – The cluster's ability to pull images from the specified Azure Container Registries.
- **Key Vault Secrets User** – The cluster's ability to read secret contents from a Key Vault.
- **Monitoring Metrics Publisher** – The cluster's ability to send metrics to Azure Monitor.
- **Network Contributor** – The cluster's ability to control the spoke virtual network.

Networking

4



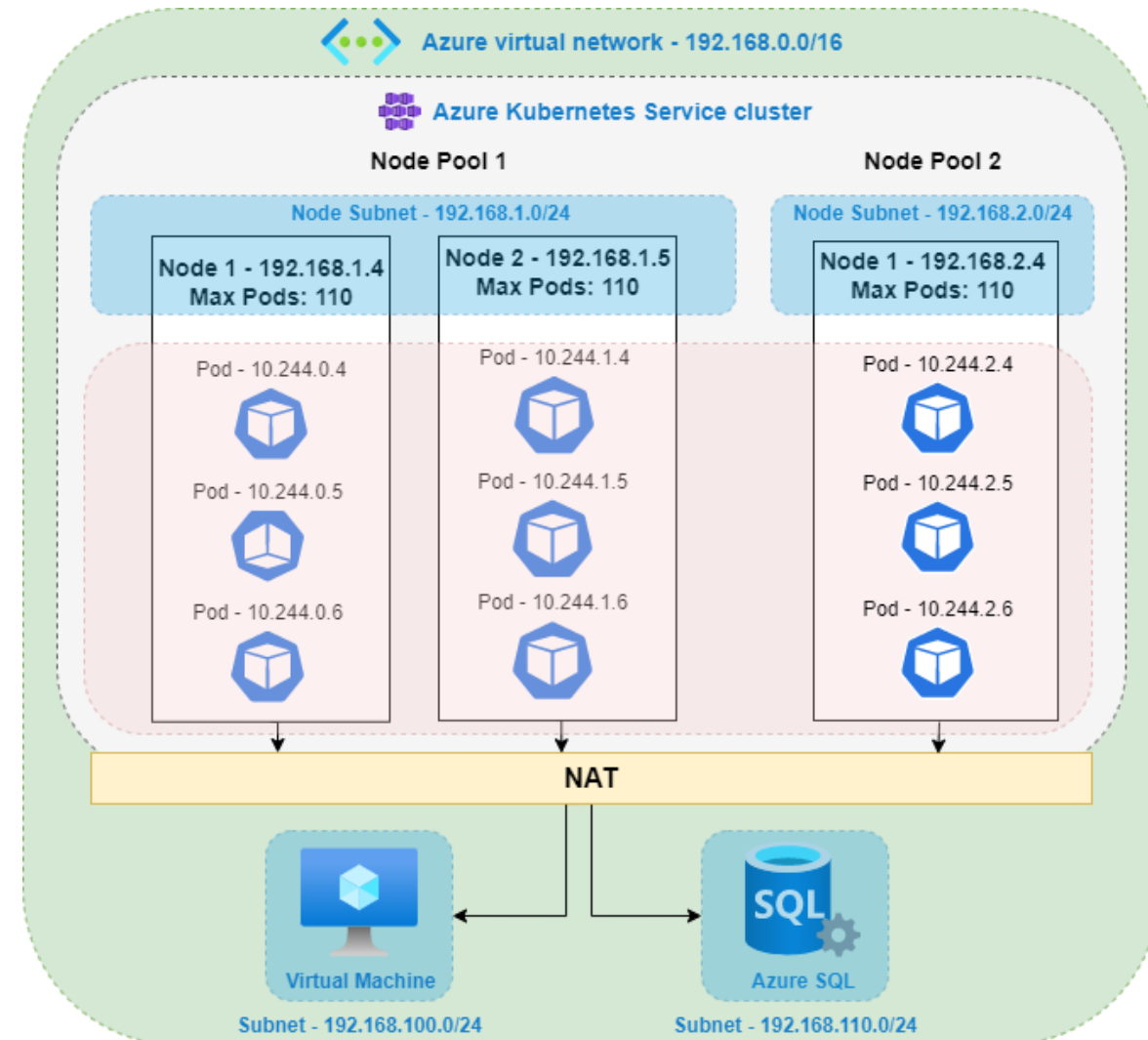
Azure Virtual Networks

There are multiple network plugins available when creating a new AKS Cluster:

- **Kubenet** networking (Basic) – Should be avoided
 - By default, AKS clusters use **kubenet**, and an Azure virtual network and subnet are created for you.
 - Pod IP addresses allocated from an internal/separate subnet (outside cluster's VNet).
- **Azure Container Networking Interface (CNI)** networking plugin
 - The AKS cluster is connected to existing virtual network resources and configurations.
 - Comes in one of 3 versions:
 - **Azure CNI Dynamic** – Pod IP addresses are allocated from a separate subnet in VNet.
 - **Azure CNI Overlay** - Pod IP addresses are allocated from an internal/separate subnet.
 - **Azure CNI Powered By Cilium** – CNI + Cilium
- **Bring your own Container Network Interface (CNI) plugin**

Kubenet (basic) Networking

- AKS virtual network and subnet are created automatically if not specified.
- Pods receive IP addresses from an internal address space, **logically different from the Nodes' VNet** (default: 10.244.0.0/16)
- Pods are **NOT** directly accessible from outside the cluster.
- Requires route tables and UDRs on cluster subnet for pod networking, which adds latency to pod communications.
- Clusters limited to 400 nodes.
- Only works with Linux nodes.
- Not the recommended option

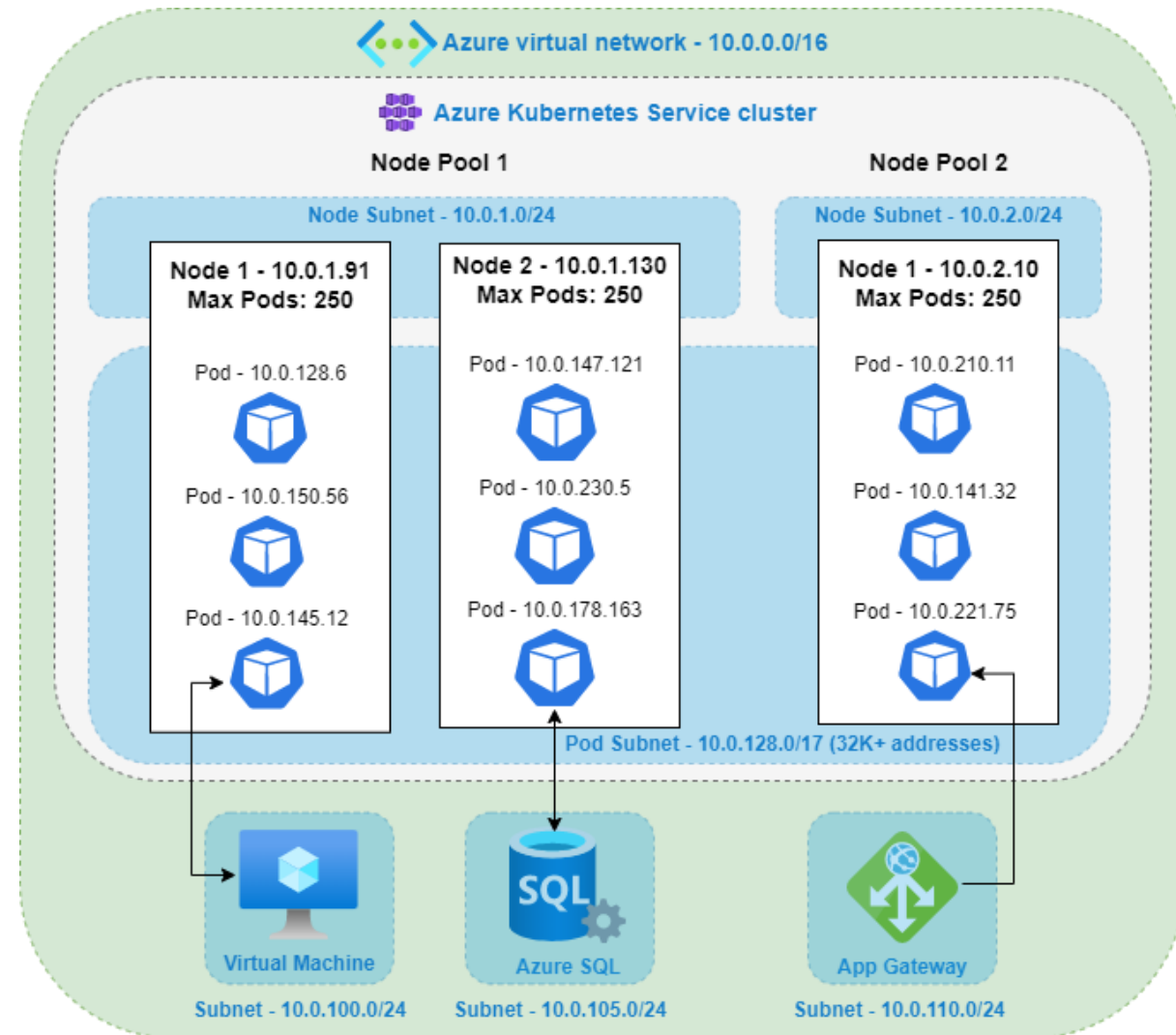


Simple Networking - Azure CNI Overlay

- **Azure CNI Overlay** works the same way as *Kubenet*
 - Pods receive IP addresses from an internal address space, **logically different from the Nodes' VNet**
 - Pods are **NOT** directly accessible from outside the cluster.
 - External access goes through a NAT layer
- However, **Azure CNI Overlay** has many advantages over *Kubenet*:
 - Supports larger clusters: max 5000 nodes and 250 pods/node
 - Simple network configuration: no additional configuration required for pod networking (no route tables or UDR required)
 - Pod connectivity performance on par with VMs in a VNet
 - Supports several Kubernetes Network Policy plugins: Azure, Calico, Cilium
 - Supports both Linux and Windows Server 2022 nodes
 - Recommended configuration for most production clusters.

Advanced Networking - Azure CNI with Dynamic IP

- **Azure CNI with Dynamic IP** allows Pods to be allocated IP addresses from a **separate subnet** within the cluster's VNET.
- Pods get IP addresses from the subnet and are **directly accessible from other devices** on the same the VNET.
- IP addresses must be unique across the network space.
- The same Pods subnet can be used on multiple clusters.
- Requires planning but is less likely to lead to IP address exhaustion because no IPs are pre-allocated and possibly "wasted".



Azure CNI Vnet – Static Block Allocation (Preview)

- Solves the Dynamic IP Allocation scalability issue allowing scaling beyond /16.
- Better IP Scalability
- Flexibility
- High performance
- Separate Vnet policies per pod
- Kubernetes network policies

Azure CNI Powered by Cilium

- Combines Azure CNI Control plane with the Cilium Data plane
- Azure CNI + Cilium leverages eBPF linux kernel features
 - Similar features to regular CNI and CNI overlay
 - Faster service routing
 - Better observability of traffic in cluster
 - Built-in network policy with Cilium
- Deployment
 - 2 Models
 - CNI overlay
 - Dynamic
- Limitations
 - Linux Only
 - Cilium L7 policy disabled
 - Hubble Disabled
 - Install not customizable for Cilium

Bring Your Own CNI plugin with AKS

- Advanced users of AKS may wish to utilize the same CNI plugin used in on-premises Kubernetes environments or to make use of specific advanced functionality available in other CNI plugins not available in Azure CNI.
- Certain requirements must be met for AKS to work with an external CNI Plugin:
 - The virtual network for the AKS cluster must allow outbound internet connectivity.
 - AKS clusters may not use **169.254.0.0/16**, **172.30.0.0/16**, **172.31.0.0/16**, or **192.0.2.0/24** for Kubernetes service address range, pod address range, or cluster virtual network address range.
 - The cluster identity used by the AKS cluster must have at least Network Contributor permissions on the subnet within your virtual network.
 - The subnet assigned to the AKS node pool cannot be a delegated subnet.
 - AKS doesn't apply Network Security Groups (NSGs) to its subnet and will not modify any of the NSGs associated with that subnet.
 - Microsoft support will not be able to assist with CNI-related issues in BYOCNI clusters

Selecting the Right Network Plugin

- Choosing which network plugin and AKS cluster will use can **only be done when the cluster is created** and cannot be changed afterwards.
- Choose **Kubenet/Azure CNI Overlay** when:
 - You have limited IP address space.
 - Most of the pod communication is within the cluster.
 - You don't need advanced AKS features such as virtual nodes
 - Need a quick cluster for development (without Windows nodes for Kubenet)
- Choose **Azure CNI Dynamic** when:
 - You have available IP address space.
 - Resources outside the cluster need to reach pods directly.
 - Most of the pod communication is to resources outside of the cluster.
 - You need AKS advanced features such as virtual nodes or Azure Network Policy.
 - You plan on using Virtual Nodes.

Planning IP Addresses

- The address space of the virtual network should be large enough to hold all subnets.
- Account for all entities that will receive traffic. IP addresses for those entities will be allocated from the subnet address space. Consider these points:
- ***Upgrade:***
 - During an upgrade process, AKS creates a node that temporarily hosts the pods, while the upgrade node is cordoned and drained. That temporary node is assigned an IP address from the cluster subnet.
 - For pods, you might need additional addresses depending on your strategy. For rolling updates, you'll need addresses for the temporary pods that run the workload while the actual pods are updated.
 - If you use the ***Recreate*** strategy, old pods are removed, and then new ones are created. So, addresses associated with the old pods are reused.

Planning IP Addresses

- **Scalability:**
 - Take into consideration the node count of all system and user nodes and their maximum scalability limit. Suppose you want to scale out by 400%. You'll need four times the number of addresses for all those scaled-out nodes.
 - Using **Azure CNI Dynamic**, each pod can be contacted directly. So, each pod needs an individual address. Pod scalability will impact the address calculation. That decision will depend on your choice about the number of pods you want to grow.
- **Azure Private Link addresses**
 - Factor in the addresses that are required for communication with other Azure services over Private Link. Addresses assigned for the links to Azure Container Registry and Key Vault.
 - Certain addresses are reserved for use by Azure. They can't be assigned.

For more information about planning IP for an AKS cluster, see [Plan IP addressing for your cluster](#).

AKS Integrations

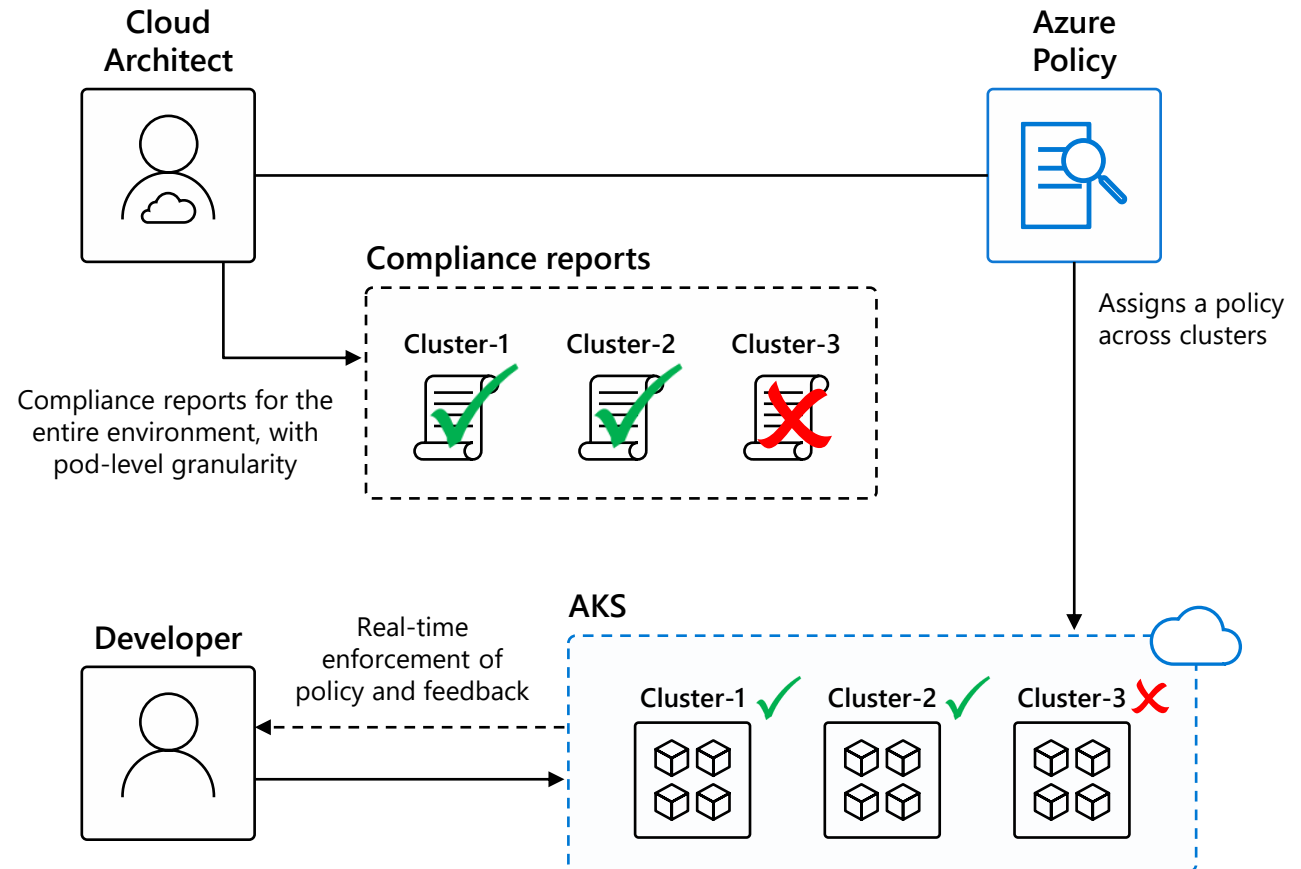


Governance with Policy

Dynamically enforce guardrails defined in [Azure Policy](#) across multiple clusters—nodes, pods, and even container images.

Apply individual policy definitions or groups of policy definitions called initiatives (sometimes called policysets) to your cluster.

Track and validate at the time of deployment or as part of CI/CD workflows



Use Azure Policy to Secure AKS Cluster

- To improve the security of your Azure Kubernetes Service (AKS) cluster, you can apply and enforce built-in security policies on your cluster using **Azure Policy**.
- Azure Policy helps to enforce organizational standards and to assess compliance at-scale.
- After installing the Azure Policy Add-on for AKS, you can apply individual policy definitions or groups of policy definitions called initiatives (sometimes called policysets) to your cluster.

Azure Policy built-in definitions for AKS

Below is a very partial list of some of the built-in Azure Policy definitions for AKS.

Name	Description	Effect(s)
Kubernetes cluster pods should use specified labels	Use specified labels to identify the pods in a Kubernetes cluster. This policy is generally available for Kubernetes Service (AKS), and preview for AKS Engine and Azure Arc enabled Kubernetes. For more information, see https://aka.ms/kubepolicydoc .	audit, deny, disabled
Kubernetes cluster containers CPU and memory resource limits should not exceed the specified limits	Enforce container CPU and memory resource limits to prevent resource exhaustion attacks in a Kubernetes cluster. This policy is generally available for Kubernetes Service (AKS), and preview for AKS Engine and Azure Arc enabled Kubernetes. For more information, see https://aka.ms/kubepolicydoc .	audit, deny, disabled
Kubernetes cluster containers should only listen on allowed ports	Restrict containers to listen only on allowed ports to secure access to the Kubernetes cluster. This policy is generally available for Kubernetes Service (AKS), and preview for AKS Engine and Azure Arc enabled Kubernetes. For more information, see https://aka.ms/kubepolicydoc .	audit, deny, disabled
Kubernetes cluster containers should only use allowed images	Use images from trusted registries to reduce the Kubernetes cluster's exposure risk to unknown vulnerabilities, security issues and malicious images. This policy is generally available for Kubernetes Service (AKS), and preview for AKS Engine and Azure Arc enabled Kubernetes. For more information, see https://aka.ms/kubepolicydoc .	audit, deny, disabled

See [Azure Policy built-in definitions for AKS](#) for a complete list of AKS policy and initiative definitions.

Azure Policy Walkthrough

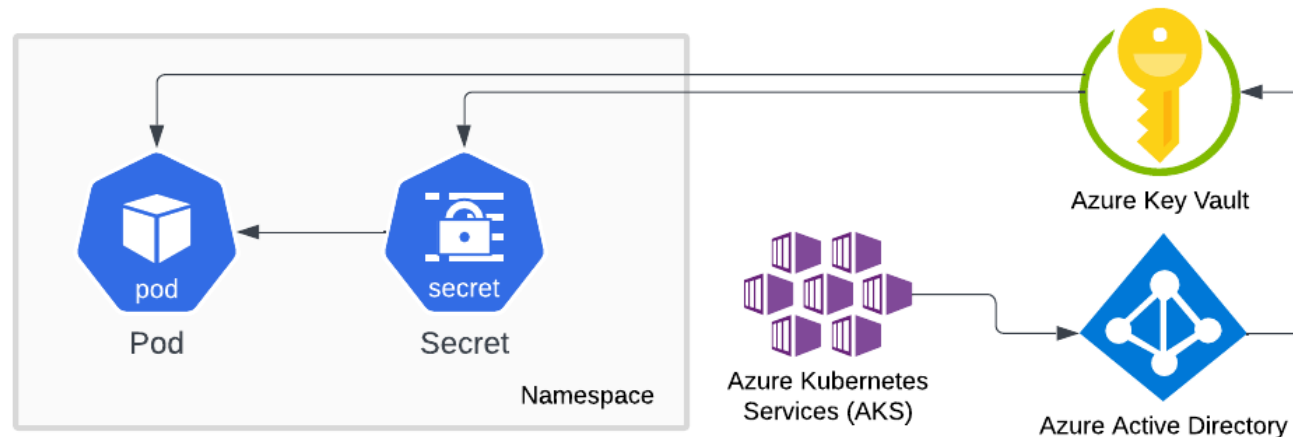
Explore Kubernetes Specific Azure Policies

AKS Network Policy

- Network policy is a Kubernetes feature available in AKS that lets you control the traffic flow between pods.
- Azure provides two ways to implement network policy. You choose a network policy option when you create an AKS cluster. The policy option can't be changed after the cluster is created:
 - Azure's own implementation, called **Azure Network Policies**.
 - **Calico Network Policies**, an open-source network and network security solution.
 - Cilium works with AKS clusters that use Azure CNI powered by Cilium
- Both implementations use Linux IPTables to enforce the specified policies.
- Policies are translated into sets of allowed and disallowed IP pairs, which are then programmed as IPTable filter rules.

Azure Key Vault

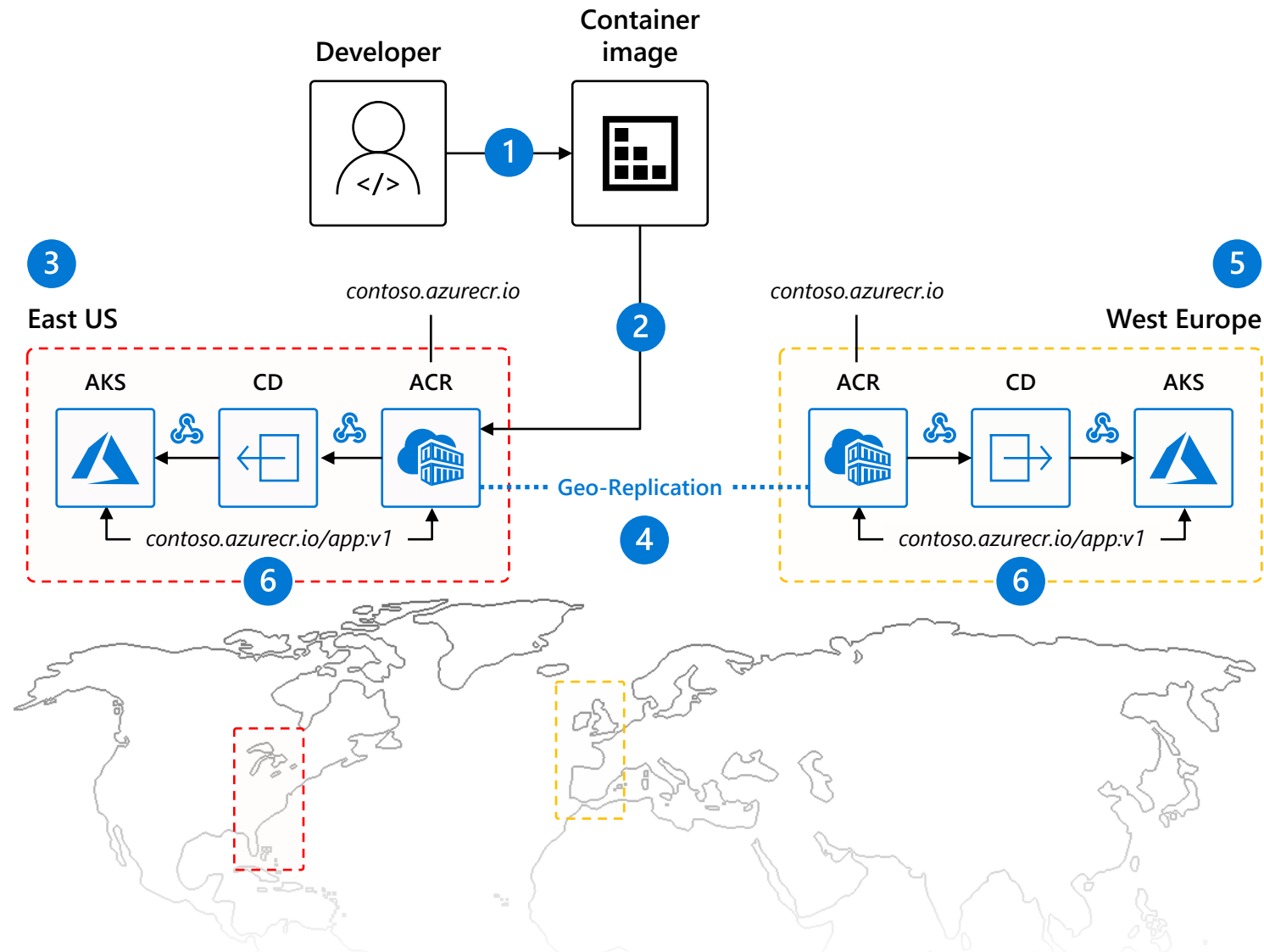
- Azure Kubernetes Service supports integration with **Azure Key Vault** to facilitate best practices when accessing sensitive information.
- A Secret Store Provider can be enabled during the creation of a cluster.
- AKS uses a managed identity to connect to Key Vault and dynamically create Kubernetes Secrets and/or inject secrets direct into containers.



Azure Container Registry

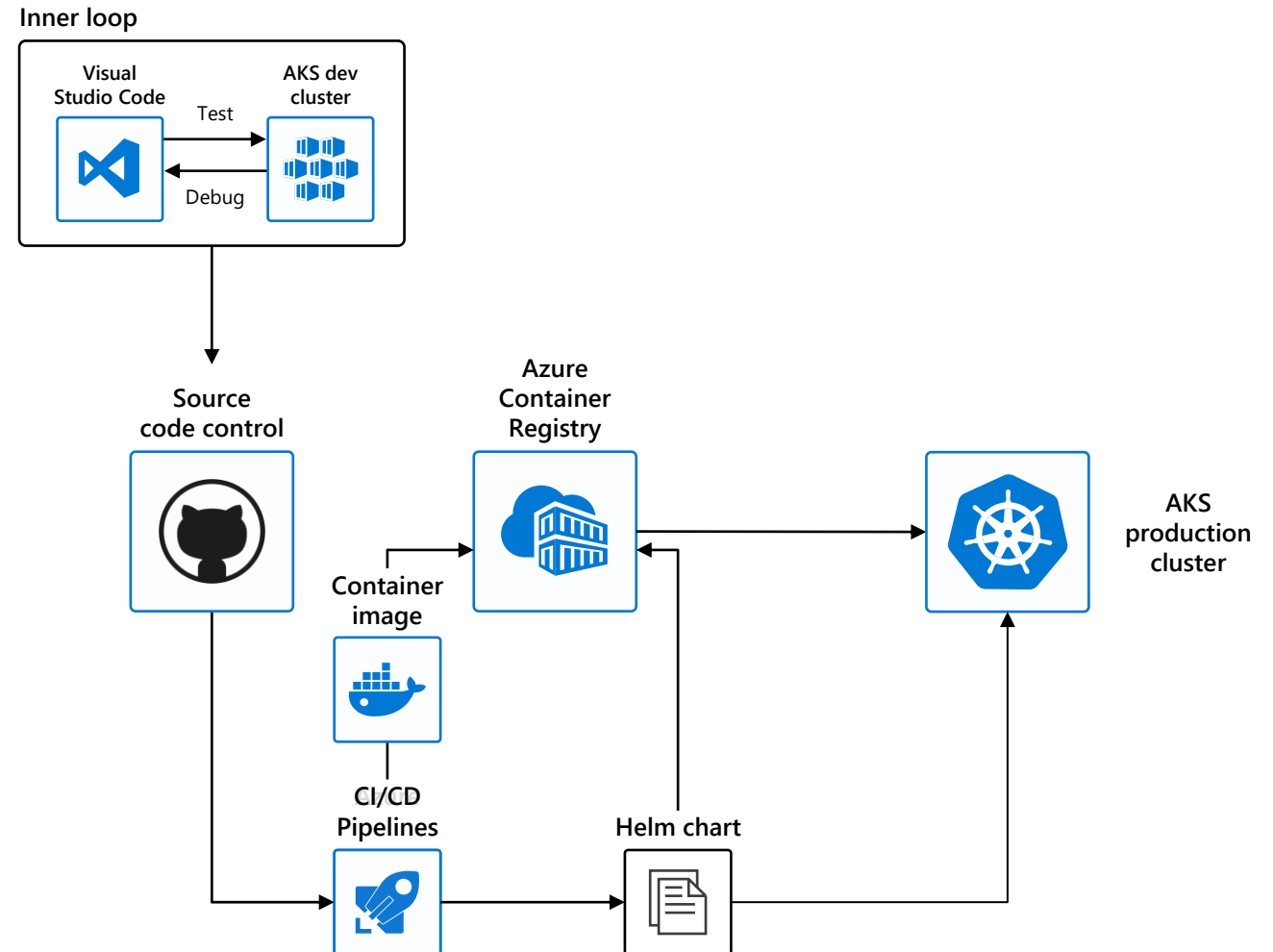
Build, store, secure, scan, replicate, and manage container images and artifacts with a fully managed, geo-replicated instance of ACR.

- Import images from public registries
- Block unauthorized access to your images.
- Reduce public facing dependencies (secure supply chain).
- You can access image pull logs to monitor activities and triage connectivity issues.
- Take advantage of integrated container scanning and image compliance using **Microsoft Defender for Containers**.



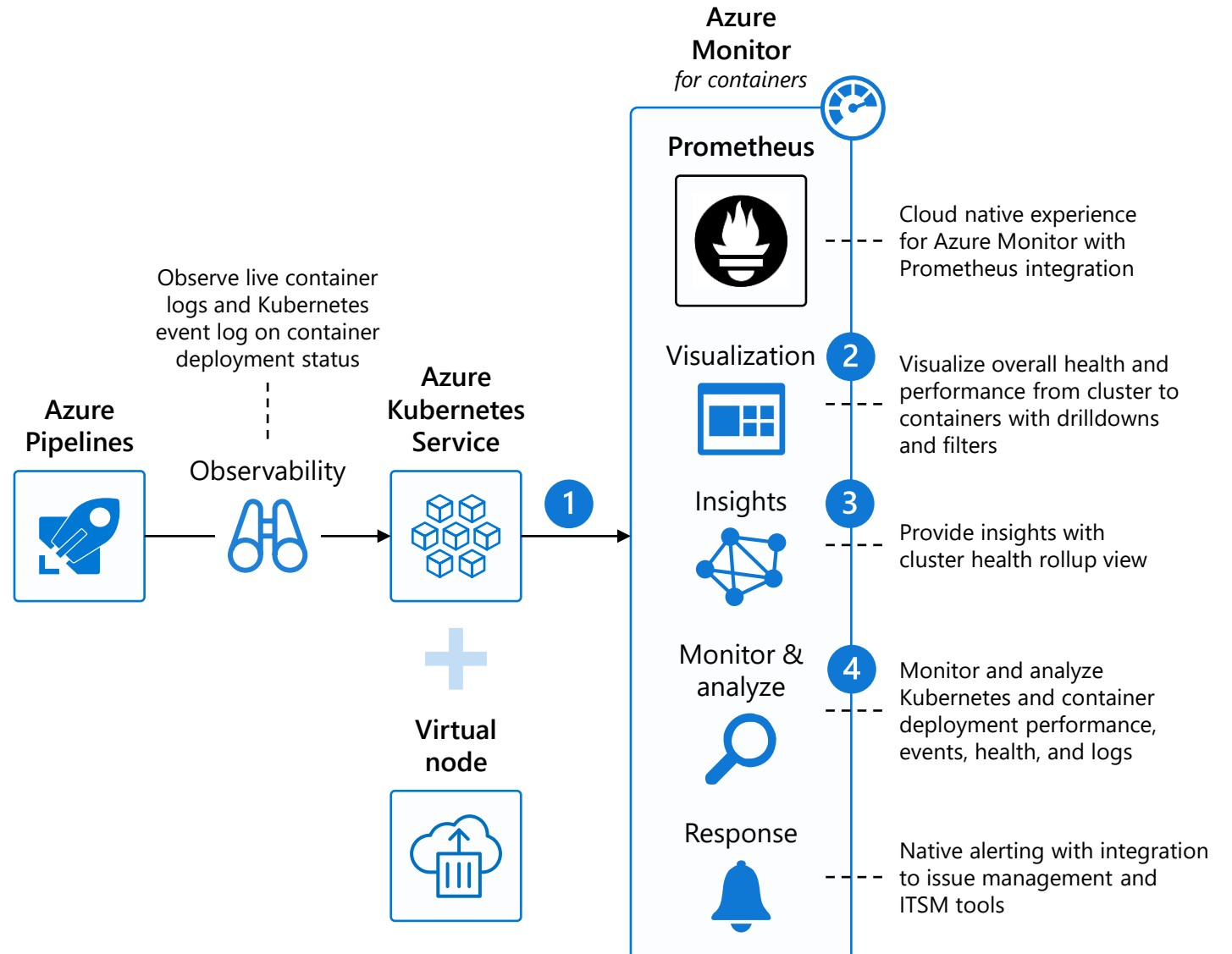
Accelerate containerized development

- Native containers and Kubernetes support in Visual Studio Code
- Private container registry with Helm support
- Develop and test Kubernetes apps without mocking up dependencies
- Effective code merge, containerization, and CI/CD pipeline with automated tasks in a few clicks
- Pre-configured canary deployment strategy
- In-depth build and delivery process review and integration testing



Azure Monitor

1. Get detailed insights about your workloads with Azure Monitor
2. Filter for details about nodes, controllers, and containers
3. See graphical insights about clusters
4. Pull events and logs for detailed activity analysis



Azure Monitor Demo

Create a Log Analytics Workspace to Send AKS Metrics and Logs to.

Prometheus and Grafana

- Use a Managed Grafana in Azure or Self hosted Grafana to view metrics
- Allows Prometheus metrics collection to be used in Azure monitor or Grafana.
- Compliments the current metrics being used by Container Insights
- Enable the AKS Prometheus Add on via the Portal or CLI
 - Does require an Azure Monitor Workspace to be enabled



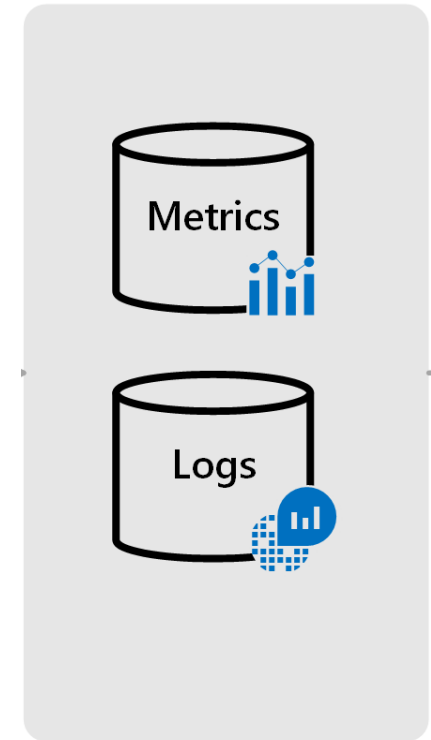
Prometheus and Grafana Demo

Create a Log Analytics Workspace to Send AKS Metrics and Logs to.

Log Analytics Workspace

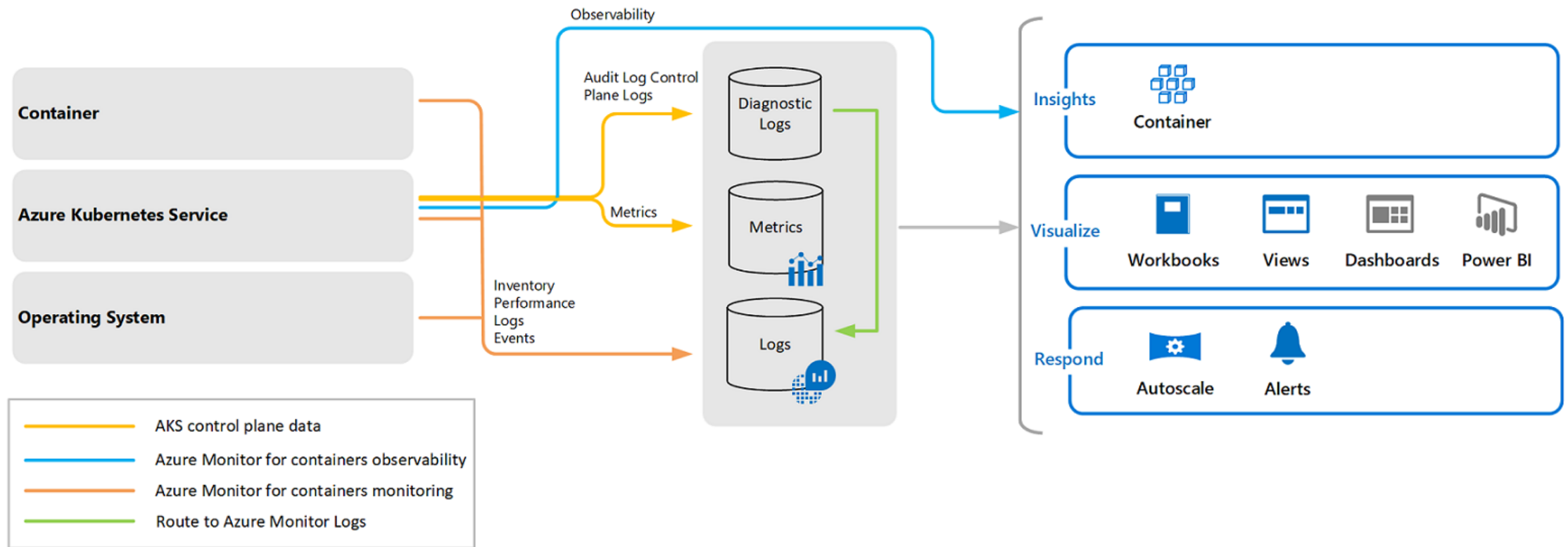
- A **Log Analytics Workspace** is a centralized databases that stores metrics and logs from AKS clusters.
- You can create a Log Analytics Workspace when you create an AKS cluster or leverage an existing instance.
- Log Analytics Workspace instances are identified by a **Workspace ID**.
- OMS Agents (Pods created on each Node by a DaemonSet) monitor nodes and send telemetry data to a Log Analytics Workspace instance*.

***Note**: Node metrics are not available from Virtual Nodes.



Azure Container Insights

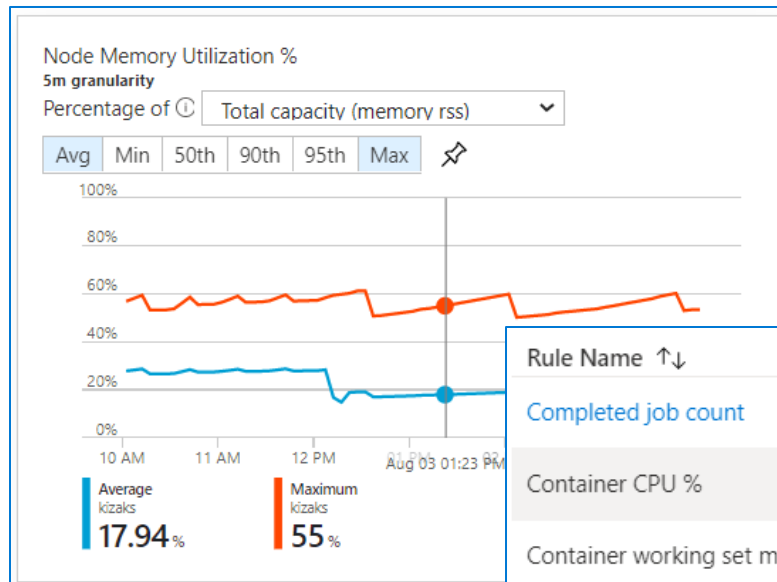
- Azure **Container Insights** is a part of Azure Monitor and integrates with AKS.



- See [How to query logs from Container Insights](#) for sample Kusto queries.

Azure Container Insights Dashboard

Container Insights allows you observe many aspects of your cluster and create alerts for critical events.



Name	Status	95th % ↓	95th	Containers	UpTime	Controller
aks-agentpool-35127589-vmss000002	Ok	6%	116 mc	17	22 days	-
Other Processes	-	0%	39 mc	-	-	-
konnnectivity-agent-5fbbb799b6-ptm72	Ok	2%	2 mc	1	18 days	konnnectivity-agen...
clusterinfo-6fb57fff9b-wz9dc	Ok	2%	32 mc	1	22 days	clusterinfo-6fb57f...
					1 day	omsagent
					6 days	stats-queue-5c9d...
					18 days	keda-operator-m...

Rule Name ↑↓	Condition ↑↓	Action Groups ↑↓	Status ↑↓
Completed job count	Number of completed jobs(more than 6 hours ...	No action group assigned	Disabled
Container CPU %	Average container CPU is greater than 95%	No action group assigned	Disabled
Container working set memory %	Average container working set memory is grea...	No action group assigned	Disabled
Failed Pod counts	Number of Pods in Failed state are greater tha...	No action group assigned	Disabled
Node CPU %	Average node CPU is greater than 80%	No action group assigned	Disabled
Node Disk Usage %	Average disk usage for a node is greater than ...	No action group assigned	Disabled
Node NotReady status	Number of nodes in not ready state are greate...	No action group assigned	Disabled

Azure Kubernetes Service Demo

Create an Azure Kubernetes Service Instance using the Azure Portal

PRO TIP: Stop/Start an AKS Cluster

- AKS workloads may not need to run continuously (for example a development cluster that is used only during business hours).
- To optimize costs during these periods, you can completely turn off (stop) an AKS cluster, which will delete all its Nodes.
- Stopping a cluster saves on all the compute costs, while maintaining all objects and cluster state for when it starts up again.
- **Example**: To stop/start a cluster:

```
az aks stop/start --name mycluster --resource-group mygroup
```

NOTE: Repeatedly starting/stopping an AKS cluster may result in errors. Once a cluster is stopped, wait at least 15-30 minutes before starting it up again.

Lab – Module 2

Create an Azure Kubernetes Service Cluster





Thank you